



Univerzitet u Beogradu  
Matematički fakultet

## Seminarski rad iz Verifikacije softvera

# Clang semantička provera: Čiste i konstantne funkcije

**Profesor i asistent:**

prof. dr Milena Vujošević Janičić  
Ivan Ristović

**Studenti:**

Đurđa Milošević 1008/2025  
Anja Milutinović 1011/2025

**Datum:** 2026.

# 1 Opis problema

Savremeni C i C++ kompajleri podržavaju različite funkcijske atribute kojima programer može eksplicitno da opiše osobine funkcija. Ove informacije omogućavaju kompajleru da primeni optimizacije.

Dva atributa ove vrste koja su se izučavala u ovom radu su **pure** i **const**. Oba atributa opisuju funkcije koje nemaju sporedne efekte (eng. *observable side effects*), ali se razlikuju po tome koliko je funkciji dozvoljeno da pristupa memoriji i od čega sme da zavisi njena povratna vrednost.

Ove informacije kompajler koristi za optimizacije zasnovanih na pretpostavci da će funkcija za iste ulazne argumente vratiti isti rezultat. Atributi **pure** i **const** prvenstveno služe kao informacije koje programer prosleđuje kompajleru radi optimizacije programa. Kompajler ne proverava da li implementacija zaista zadovoljava uslove koje atribut nameće, već pretpostavlja da je deklaracija tačna i na osnovu nje primenjuje odgovarajuće optimizacije. Zbog toga pogrešno označavanje funkcije može dovesti do neispravnih optimizacija i promenjenog ponašanja programa.

## 1.1 Pure funkcije

Funkcija označena atributom **pure** ne sme da menja stanje programa koje je vidljivo ostatku programa. Jedini rezultat njenog izvršavanja sme biti povratna vrednost funkcije. Funkcija sa atributom **pure** sme da čita proizvoljne nepromenljive (eng. *non-volatile*) objekte iz memorije, uključujući globalne promenljive i objekte do kojih dolazi preko pokazivača ili referenci. Međutim, nije dozvoljeno da ih menja niti da izvršava bilo kakve druge operacije koje proizvode sporedne efekte. Ukoliko se vrednosti koje funkcija čita promene između dva poziva, može se promeniti i rezultat funkcije, pa se optimizacije mogu primeniti samo dok kompajler može da garantuje da se relevantno stanje nije promenilo. [1]

## 1.2 Const funkcije

Atribut **const** predstavlja strožiju verziju atributa **pure**. Pored toga što funkcija ne sme da proizvodi sporedne efekte, njena povratna vrednost sme da zavisi isključivo od vrednosti njenih argumenata. Odnosno, funkcija ne sme da čita promenljive čija bi promena mogla da utiče na rezultat funkcije. Dozvoljeno je jedino čitanje objekata koji predstavljaju konstante i čija vrednost ne može da se promeni tokom izvršavanja programa. Zbog toga se ovakav atribut uglavnom ne koristi za funkcije koje primaju pokazivače ili

reference, jer sadržaj memorije na koji oni pokazuju može da se promeni između dva poziva funkcije. [1]

| Osobina                                     | pure | const |
|---------------------------------------------|------|-------|
| Nema sporednih efekata                      | ✓    | ✓     |
| Može da čita globalne promenljive           | ✓    | ×     |
| Može da menja vrednost globalne promenljive | ×    | ×     |
| Može da čita preko pokazivača/referenci     | ✓    | ×     |
| Može da piše preko pokazivača/referenci     | ×    | ×     |

Tabela 1: Poređenje atributa `pure` i `const`

### 1.3 Clang Static Analyzer

Clang Static Analyzer (CSA) predstavlja alat za statičku analizu programa koji je deo LLVM/Clang kompajlerske infrastrukture. Analiza se zasniva na simboličkom izvršavanju (engl. *symbolic execution*), pri čemu se istovremeno razmatraju različiti tokovi izvršavanja i održava apstraktno stanje programa koje opisuje vrednosti promenljivih, sadržaj memorije i druge relevantne informacije.

U ovom radu Clang Static Analyzer korišćen je za implementaciju proveravača koji analizira funkcije označene atributima `pure` i `const`. Tokom simboličkog izvršavanja proveravač prati da li funkcija proizvodi sporedne efekte, kao i da li pristupa memoriji na način koji nije dozvoljen za odgovarajući atribut.

## 2 Opis arhitekture sistema

djurjda  
checker - callback programstate bugreport pbjasniti i maske (util) korišćenje

## 3 opis rešenja problema (osnovne ideje, obrazloženja za ključne odluke, opis osnovnog algoritma)

specifčno

### 3.1 pure checker

– nabrojati provere koje proveravamo (sa primerima?) anja – objasniti program state anja – objasniti po callbackovima djurdja – objasniti da je sve path sensitive sa primerom anja

### 3.2 const checker

djurjda – objasniti da li je razdvojeno i da li jeste ili nije i zasto – objasniti razliku u odnosu na pure

## 4 Zaključak

djurjda

## Literatura

- [1] GNU Project. *Common Function Attributes*. GNU Compiler Collection (GCC). Sections “pure” and “const”, Accessed: 2026-07-26.